❶ EmailSender is registered with the container.

❷ There's already an IMessageSender implementation, so SmsSender isn't registered.

Code like this doesn't often make a lot of sense at the application level, but it can be useful if you're building libraries for use in multiple apps. The ASP.NET Core framework, for example, uses `TryAdd*` in many places, which lets you easily register alternative implementations of internal components in your own application if you want.

You can also replace a previously registered implementation by using the `Replace()` extension method. Unfortunately, the API for this method isn't as friendly as the `TryAdd` methods. To replace a previously registered `IMessageSender` with `SmsSender`, you'd use

```
builder.Services.Replace(new ServiceDescriptor(
    typeof(IMessageSender), typeof(SmsSender), ServiceLifetime.Scoped
));
```

TIP When using `Replace`, you must provide the same lifetime that was used to register the service that's being replaced.

We've pretty much covered registering dependencies but touched only vaguely on one important aspect: lifetimes. Understanding lifetimes is crucial in working with DI containers, so it's important to pay close attention to them when registering your services with the container.

9.4 Understanding lifetimes: When are services created?

Whenever the DI container is asked for a particular registered service, such as an instance of `IMessageSender`, it can do either of two things to